

Java

Abstraction, interface, final keyword

Theory

1. Abstraction

Definition: Abstraction is the process of hiding implementation details and showing only essential features. **Use case:** Helps achieve design flexibility and reduces complexity. **Key point:** Achieved in Java using **abstract classes** and **interfaces**.

Syntax:

```
abstract class Example {  
    abstract void abs(); // abstract method  
    void display() { System.out.println("Non-abstract method"); }  
}
```

2. Interfaces

Definition: An interface is a contract that contains only abstract methods (Java 8+ allows default and static methods). **Use case:** Supports **multiple inheritance** and **full abstraction**. **Key point:** A class implements an interface and must provide definitions for all its methods.

Syntax:

```
interface Inter {  
    void display();  
    void run();  
}  
  
class Demo implements Inter {  
    public void display() { ... }  
    public void run() { ... }  
}
```

3. Final Keyword

Definition: The final keyword restricts modification. **Use case:**

- **Final variable** → value cannot be changed.
- **Final method** → cannot be overridden.
- **Final class** → cannot be inherited.

Syntax:

```
final class FinalClass {  
    final int count = 10;  
    final void display() { System.out.println("Final method"); }  
}
```

Coding Round Questions

- Abstract class → Create an abstract class with one abstract and one non-abstract method.
- Interface implementation → Define an interface with two methods and implement it in a class.
- Multiple inheritance → Demonstrate multiple inheritance using two interfaces.
- Final variable → Show how a final variable cannot be reassigned.
- Final method → Show how a final method cannot be overridden.
- Final class → Show how a final class cannot be inherited.

Challenge

Problem: Payment Gateway System

Requirements:

- Create an **abstract class** Payment with an abstract method pay() and a non-abstract method successMessage().

- Create subclasses: UPI, Card, COD that implement pay() differently.
- Create an **interface** Refundable with method refund().
- Implement Refundable in Card and UPI classes.
- Use final keyword to define a constant TRANSACTION_FEE that cannot be changed.
- Demonstrate polymorphism by calling pay() and refund() methods.

```
abstract class abstractExample {

    abstract void abs(); // incomplete method

    void display(){
        System.out.println("This is non abstract method ");
    }

}

class abstractChild extends abstractExample {

    void abs(){
        System.out.println("Abstract child method ");
    }

}

class abstractChild1 extends abstractExample{

    void abs(){
        System.out.println("Abstract child 1 method");
    }

}

public class abstraction {

    public static void main(String[] args) {
        System.out.println("Abstraction");

        // java doesn't allow to create object for abstract class
        // abstractExample obj = new abstractExample();
        // obj.abs();
    }
}
```

```

        abstractChild obj = new abstractChild();
        obj.abs();

        abstractChild1 obj1 = new abstractChild1();
        obj1.abs();

    }

}

```

```

interface inter{
    public void display();
    void run();
}

interface outer{
    void data();
}

class demo{

}

//hybrid inheritance (combination of class and interfce )
class HybridInheritance extends demo implements outer{

    public void data(){
        System.out.println("Data method from outer interafce ");
    }
}

// multiple inheritance (one child -> multiple parent)
class multipleInheritance implements inter, outer{
    public void display(){
        System.out.println("display method from multiple inheritance ");
    }

    public void run(){
        System.out.println("run method from multiple inheritance ");
    }

    public void data(){
        System.out.println("data from outer interface");
    }
}

class abstractClass implements inter{

```

```

    public void display(){
        System.out.println("display method from abstractClass");
    }

    public void run(){
        System.out.println("run method from abstractClass");
    }
}

public class Interface {
    // in interface we can achieve full abstraction
    public static void main(String[] args) {

        System.out.println("Interface");
        abstractClass obj = new abstractClass();
        obj.display();
        obj.run();

    }
}

```

```

final class finalClass {

    final int count = 10;
    final void display(){
        System.out.println("final class method");
    }
}

class finalClassChild extends finalClass{

    // void display(){
    //     System.out.println("finalclass child");
    // }

}

// final (field)=> we don't able to change the value of it
// final (method)=> we can't override in subclass
// final (class) => we can't inherit that class

public class FinalKey {

    public static void main(String[] args) {

```

```
        System.out.println("Hello there");

        finalClass obj = new finalClass();

        // obj.count = 20;
        System.out.println(obj.count);

        finalClassChild obj1 = new finalClassChild();

        obj1.display();
    }
}
```